

第 4 组接口文档

模块：工具注册 + OS Skills（工具服务层） 对应 UF0² 组件：Tools | 所属层次：工具服务层
| OS 原理关联：持久性（文件系统）

- 课程：操作系统设计 • Linux 桌面 Agentic OS
- 第 4 组：工具服务层（ToolRegistry + SkillLibrary）
- 版本：v1.1（2026-09-11，接口稳定 + 安全硬拦截接入后）
- 对应检查点：检查点 1（接口文档 + Mock 代码）、检查点 2（模块代码 + 接口落定）

一、接口设计总则

第 4 组向上层（第 1/2/3 组）只暴露一份工具清单 + 一个调用入口 + 一种返回结构，全部接口接收结构化的 `(name, params)`，统一返回 `tool_result_json`。

上层(Agent / 执行层 / 任务规划)

| list_*(())

| call(name, params)

▼

[第4组] ToolRegistry + SkillLibrary

└ 能力层：文件/文件夹/文本/磁盘/系统/应用/浏览器/进程/网络/环境变量

└ MockToolRegistry（同构，可整体替换，供跨组联调）

1.1 与设计书契约的一致性

- 核心返回结构 `tool_result_json` 与任务设计书第 4 组契约完全一致；
- 输入 `call(name, params)` 与第 3 组 `ToolClient` 契约对齐（`TOOL_NOT_FOUND` / `TOOL_ERROR` 语义一致）；
- 工具反序列化与 MCP `CallToolResult` 对齐（见「五、MCP 对齐」）。

二、输出接口（第 4 组 → 上层）

2.1 统一返回结构 tool_result_json

```
{
  "success": true,           // 或 false
  "result": "...",          // 成功时的结果（字符串 / 结构化对象）
}
```

```
"error": "...",          // 仅 success=false 时出现
}
```

第 4 组在契约基础上的工程扩展（保证可审计 + 可追踪）：

```
{
  "success": true,
  "result": "...",
  "tool": "organize_downloads",    // 调用的工具/技能名
  "elapsed_ms": 42,                // 耗时（供统计/审计）
  "ts": "2026-09-09T09:00:00"     // 时间戳（供第 5 组审计 / RAG）
}
```

2.2 返回结构字段说明

字段	类型	必填	说明
success	bool	☒	是否执行成功
result	str/object	成功时	执行结果（字符串或结构化对象）
error	str	失败时	错误描述（{success:false,error} 统一）
tool	str	扩展	实际执行的工具/技能名
elapsed_ms	int	扩展	调用耗时（毫秒）
ts	str	扩展	ISO8601 时间戳，供审计/RAG 入库

三、输入接口（上层 → 第 4 组）

接口	请求参数	返回	说明
list_tools()	—	工具清单	全量工具名
list_tools_meta()	—	清单[{name, description, category}]	含描述，供规划/展示
list_tools_mcp()	—	MCP 格式清单	供 MCP 兼容客户端
list_tools_openai()	—	OpenAI function 格式	直接喂给 function-calling LLM
list_tools_anthropic()	—	Anthropic 工具格式	直接喂给 Claude
list_skills()	—	技能清单	26 个组合技能
call(name, params)	name + params	tool_result_json	统一调用工具/技能

接口	请求参数	返回	说明
<code>call_safely(name, params)</code>	同上	<code>tool_result_json</code>	带风险确认门的安全调用
<code>get_stats()</code>	—	调用统计	总次数 / 各工具次数 / 最近结果 / Top5
<code>get_tool_signature(name)</code>	工具名	SHA256 签名	工具来源/完整性校验
<code>list_tool_signatures()</code>	—	全部签名	批量校验
<code>verify_tool_integrity(name)</code>	工具名	校验结果	检测注册后是否被篡改
<code>register(name, func)</code> / <code>register_tool</code>	名+实现	注册成功并自动签名	动态扩展
<code>unregister(name)</code>	工具名	注销	动态收缩

3.1 call 的分发规则

```
call(name, params)
├ name 是 agent 白名单组合技能 → 走 SkillLibrary（技能编排多层工具）
├ name 是底层工具 → 走 ToolRegistry
└ name 未知 → {success:false, error:"工具不存在: <name>"}（TOOL_NOT_FOUND）
```

四、工具目录（61 工具）

4.1 文件 / 文件夹

`copy_file`、`move_path`（别名 `move_file`）、`delete_file`、`rename_path`、`create_file`、`create_directory`、`delete_directory`、`list_directory`、`sort_directory`、`index_files`、`manage_archive`

4.2 文件元信息 / 校验

`file_exists`、`get_file_size`、`get_path_metadata`、`get_directory_metadata`、`check_file_access`、`file_checksum`、`compare_files`

4.3 文本 / 批量 / 查找

`read_text_file`、`write_text_file`、`edit_file`（片段精确替换）、`batch_copy`、`batch_move`、`search_files`、`search_in_files`（内容/正则带行号）

4.4 磁盘 / 系统 / 文件定位

disk_usage 、 directory_size 、 find_empty_files 、 find_empty_directories 、 find_old_files 、 find_large_files 、 current_directory 、 run_command （沙箱受限 + 超时）

4.5 图片 / 窗口 / 屏幕（GUI 辅助）

get_image_metadata 、 rotate_image 、 list_windows 、 find_window 、 activate_window 、 send_hotkey 、 send_text 、 take_screenshot 、 click_at

4.6 浏览器（CDP 跨平台，13 工具）

browser_launch 、 browser_close 、 browser_navigate 、 browser_snapshot 、 browser_click 、 browser_type 、 browser_read_text 、 browser_screenshot 、 browser_wait 、 browser_inspect （SoM 视觉定位）、 browser_find （结构化局部搜索）、 browser_refresh （SPA 恢复， Page.reload ）、 browser_bring_to_front （窗口前置）

浏览器视觉定位两条新增链路（详见 docs/miniyu 浏览器与桌面视觉定位增强方案（DOM-SoM-坐标混合架构）.md）： - browser_inspect （SoM）：给页面截一张“交互元素带编号 N 覆盖层”的图，Agent 看编号后用 browser_click(target='som:N') / browser_type(target='som:N', ...) 操作。编号只是截图上的视觉标签，真正点击落到当前 DOM 的稳定 ref；单一 InspectSession + URL/DOM 指纹做失效判定——页面变化后旧的 som:N 自动抛错逼重新 inspect。 - browser_find （结构化）：按目标文字/名称/角色（可加 CSS 选择器限域）局部搜索元素，大页面不必取整页结构，返回带稳定 ref 的清单；ref 可直接给 browser_click/browser_type 用。

browser_click / browser_type 统一 target 参数，兼容 ref (e...) / som:N / CSS 选择器三套寻址；browser_snapshot 同样返回 ref。坐标一律存 CSS 视口像素、落点按截图 DPR 换算成设备像素；动作后返回客观 Action Verification (url_changed / page_changed) 供 Agent 确认是否生效。

4.7 进程 / 网络 / 环境变量

list_processes 、 get_process_info 、 kill_process 、 terminate_process 、 network_status 、 ping_host 、 list_env_vars 、 get_env_var

五、技能目录（26 组合技能）

类别	技能
文件整理	organize_downloads 、 smart_organize 、 cleanup_temp 、 cleanup_by_type 、 batch_archive 、 summarize_files 、 deduplicate_files

类别	技能
查找/索引	search_file、find_large_files、find_recent_files、rebuild_index、query_index、duplicate_finder
回收站/备份	trash_file、restore_file、empty_trash、safe_delete、backup_file、restore_backup
系统/应用	system_info、app_open
通信/联网	app_send_message、read_qq_chat、send_email、browser_search、browser_extract

技能 = 多个原子工具 + 业务逻辑编排。示例 `organize_downloads("~/Downloads")`：扫描 → 按扩展名分 文档/图片/视频/音频/压缩包/代码/其他 → 逐类建子目录 → 移动 → 返回 `{success:true, result:"整理完成：共 N 个文件，分类 M 类"}`。

六、错误处理规范

错误码语义	返回	场景示例
成功	<code>{success:true, result}</code>	操作完成
工具不存在	<code>{success:false, error:"工具不存在：<name>"}</code>	未注册工具/技能（TOOL_NOT_FOUND）
参数错误	<code>{success:false, error:"缺少必要的参数：<param>"}</code>	必填参数缺失/类型错误
执行异常	<code>{success:false, error:"<异常信息>"}</code>	底层抛异常，统一归一（TOOL_ERROR）
安全拦截	<code>{success:false, error:"危险指令：<cmd>"}</code>	命中 DANGEROUS_ACTIONS 直接拒绝
用户拒绝	<code>{success:false, error:"用户拒绝了高危操作：<name>"}</code>	确认门被拒/沙箱 check_safe 拒绝

原则：所有失败统一为 `{success:false, error}`，上层（第 3 组/Agent）无需感知异常类型差异。

七、MCP 协议对齐

第 4 组不依赖 MCP serve，但接口语义与 MCP 对齐，以便未来无缝接入 MCP 客户端：

MCP 概念	第 4 组对应
<code>tools/list</code>	<code>list_tools_mcp()</code>
<code>tools/call</code> + <code>CallToolResult</code>	<code>call(name, params)</code> → <code>tool_result_json</code>
Tool Schema（name/description/inputSchema）	<code>get_tool_spec(name)</code> / <code>register</code> 时由签名自动推导

MCP 概念	第 4 组对应
资源/提示（resources/prompts）	本期不实现（工具层聚焦能力）

八、Mock 契约（跨组联调）

- `mock/mock_tools.py`：61 工具同构 Mock（与正式版同名/同参/同返回 `tool_result_json`）；
- `MockToolRegistry` / `MockSkillLibrary` / `MockOSServiceAPI` 三件套，对应各层接口；
- 仅内存实现（不改磁盘），保证其它组无真实系统也能离线联调、可安全复现。

九、与其它组的对接清单

对端组	对接物	说明
第 1 组（AI Shell/HostAgent）	工具清单 + <code>call</code>	用户指令 → 意图 → 工具能力落地
第 2 组（任务规划）	工具/技能清单	规划阶段即知有哪些能力可用
第 3 组（执行层）	<code>call(name, params)</code> → <code>tool_result_json</code>	GUI 或工具二选一时，确定性系统操作走第 4 组
第 5 组（协调/安全/RAG）	调用统计 + 审计 <code>ts</code> + 签名	审计日志与执行轨迹入库；危险指令硬拦截协同
全体（联调）	<code>MockToolRegistry</code>	无真实系统时离线复现

十、接口稳定性与测试

- 接口契约自检查点 1 定稿后无协议漂移（真机 LLM 与 Mock 同契约）；
- 583 项测试全绿，覆盖 `register/call/list/签名/统计/异常/安全拦截/浏览器视觉定位/Mock` 等；
- 接口可回归安全网由 `tests/`（25 文件）保障，`core/` 与 `mock/` 双份契约一致。

10.1 观测接口：`token_usage` 工具 + `run_log_*.jsonl`（2026-09-12 新增）

- `token_usage`（只读，Agent 自带函数）：返回 `{ summary: { turn_total, conversation_total, max_allowed, steps, images_this_turn }, per_step: [...] }`，`per_step` 每步含 `step / tool_hint / prompt_tokens / completion_tokens / total_tokens / image_tokens / image_count`。
- `run_log_*.jsonl`（每次运行一份，会话产物目录）：逐行两种事件——
- `kind=llm`：`ts`（带毫秒） / `step` / `thought`（模型思考） / `elapsed_ms` / `prompt_tokens` / `completion_tokens` / `total_tokens` / `reasoning_tokens`（思考推理 token，用于对账账单差） /

`image_tokens` / `message_count` / `image_count` / `char_count` / `cum_total_used`;

- `kind=action: ts / step / tool / args (脱敏) / ok / elapsed_ms / result (摘要)`。
- 动作参数记录会脱敏 `password/token/secret/api_key/authorization/cookie` 等字段，避免密钥落盘。

配套: `docs/第4组设计文档.md` (详细架构/时序/流程)、`docs/第4组单元测试报告.md`、`mock/mock_tools.py`。